2 marks

Q1. Design a java program to calculate area of shapes (circles, rectangle) using method overloading

Method Overloading: Same method name, different parameters. Used to perform similar operations (area calculation here).

Code:

```
class Area {
    double area(double r) {   // Circle
        return Math.PI * r * r;
    }
    double area(double l, double b) {   // Rectangle
        return l * b;
    }
    public static void main(String[] args) {
        Area obj = new Area();
        System.out.println("Circle Area: " + obj.area(5));
        System.out.println("Rectangle Area: " + obj.area(4, 6));
    }
}
```

Q2. What is the Object reference variable?

A reference variable stores the address of an object, not the object itself. Objects are created using new keyword. Multiple references can point to the same object.

Code:

```
class Box {
    int x = 10;
}
class Test {
    public static void main(String[] args) {
        Box b = new Box();  // b is reference variable
        System.out.println(b.x);
    }
}
```

Q3. Assume Box id class (with two instance variable l and b) what do you think following code fragment does ?

```
Box b1 = new Box();
Box b2 = b1;
```

Only one object is created.
b1 and b2 both refer to same object.
This is called reference assignment (aliasing).
Changes using one reference affect the other.
Code:

```
class Box {
   int value = 10;
}
class Test {
   public static void main(String[] args) {
      Box b1 = new Box();
      Box b2 = b1;
      b2.value = 50;
      System.out.println(b1.value); // Output: 50
   }
}
```

Q4. With the help of program discuss the concept of Static variables

Declared using static keyword.
Shared by all objects (common memory).
Belongs to class, not object.
Code:

```
class Counter {
   static int count = 0;
   Counter() {
      count++;
      System.out.println("Count: " + count);
   }
   public static void main(String[] args) {
      new Counter();
      new Counter();
      new Counter();
   }
}
```

Output:
Count: 1
Count: 2
Count: 3

4 marks

Q1. StudentData class that demonstrates the use of constructors, this keyword and method overloading.

Theory:
Constructor initializes object.
this refers to current object.
Method overloading: same method name, different parameters.

Code:

```
class StudentData {
```

```
    String name;
    int age;
    // Default constructor
    StudentData() {
        name = "Unknown";
        age = 0;
    }
    // Parameterized constructor
    StudentData(String name, int age) {
        this.name = name;
        this.age = age;
    }
    // Method overloading
    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
    void display(String msg) {
        System.out.println(msg + " Name: " + name + ", Age: " + age);
    }
    public static void main(String[] args) {
        StudentData s1 = new StudentData();
        StudentData s2 = new StudentData("Jaspreet", 22);
        s1.display();
        s2.display("Student Info:");
    }
}
```

Q3. Explain Access Specifiers in Java with examples. Differentiate between private, public and protected access modifiers.

Q4. A company wants to maintain Employee details using java. You are required to create a java program that includes : class Employee with attributes like name, age and salary. A method to calculate annual salary. Write a program and explain how constructor and method overloading are used in your solution.

Theory:
Class stores data (name, age, salary).
Constructor initializes values.
Method calculates annual salary.
Code:

```
class Employee {
    String name;
    int age;
    double salary;
    Employee(String name, int age, double salary) {
        this.name = name;
```

```
        this.age = age;
        this.salary = salary;
    }
    double annualSalary() {
        return salary * 12;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Annual Salary: " + annualSalary());
    }
    public static void main(String[] args) {
        Employee e = new Employee("Aman", 30, 50000);
        e.display();
    }
}
```

Q5. Explain Constructors and its types in java with examples.

Theory:
A constructor is a special method used to initialize objects.
Types:
1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor (user-defined in Java)

Code:

```
class Box {
    int length;
    // Default
    Box() {
        length = 0;
    }
    // Parameterized
    Box(int l) {
        length = l;
    }
    // Copy
    Box(Box b) {
        length = b.length;
    }
    void show() {
        System.out.println("Length: " + length);
    }
    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box(10);
        Box b3 = new Box(b2);
```

```
        b1.show();
        b2.show();
        b3.show();
    }
}
```

Q6. Difference between constructor and method

Theory:

Feature	Constructor	Method
Name.	Same as class	Any name
Return Type	No return type	Must have return type
Purpose	Initialize object	Perform operation
Call	Called automatically.	Called explicitly

Example Code:

```
class Test {
    Test() {  // Constructor
        System.out.println("Constructor called");
    }
    void show() { // Method
        System.out.println("Method called");
    }
    public static void main(String[] args) {
        Test t = new Test();
        t.show();
    }
}
```

8 marks

Q1. Design and implement a Java program that demonstrates the basic structure of a java program using a class with constructors-
Create a class Student with data members - RollNo, name, obtained_marks, maximum_marks of a semester and a compute method to calculate percentage of marks of a semester.
Marks use of a parametrized constructor, object as parameter for a compute method.

A class is a blueprint for objects.
Data members store information (rollNo, name, marks).
A parameterized constructor initializes values at object creation.
A method performs operations (here: percentage calculation).
Passing object as parameter allows methods to use object data.
Code:

```
class Student {
    int rollNo;
    String name;
```

```
    double obtainedMarks;
    double maxMarks;
    // Parameterized Constructor
    Student(int rollNo, String name, double obtainedMarks, double maxMarks) {
        this.rollNo = rollNo;
        this.name = name;
        this.obtainedMarks = obtainedMarks;
        this.maxMarks = maxMarks;
    }
    // Compute method (object as parameter)
    static void compute(Student s) {
        double percentage = (s.obtainedMarks / s.maxMarks) * 100;
        System.out.println("Roll No: " + s.rollNo);
        System.out.println("Name: " + s.name);
        System.out.println("Percentage: " + percentage + "%");
    }
    public static void main(String[] args) {
        Student s1 = new Student(101, "Jaspreet", 450, 500);
        compute(s1);
    }
}
```

Q 2. Garbage Collection + finalize() in detail

Explain garbage collection and finalize indetail

```
class Sample {
    protected void finalize() {
        System.out.println("Garbage collected");
    }

    public static void main(String[] args) {
        Sample s = new Sample();
        s = null;
        System.gc();
    }
}
```